

Existing Pashto Translation Models, Resources, and Improvement Plan for Neural Machine Translation

Hrithik Sharma

May 2026

Abstract

This article presents a detailed study of existing machine translation models, datasets, and improvement strategies for Pashto language translation. The current baseline system uses the pretrained `facebook/nllb-200-distilled-600M` model for Pashto-to-English, direct Pashto-to-Hindi, and Pashto-to-English-to-Hindi pivot translation. Since Pashto is a low-resource language, translation quality can be improved through high-quality dataset collection, noise filtering, fine-tuning, back-translation, model comparison, and proper evaluation. This article also discusses the possibility of training a translation model from scratch by creating a new Pashto-English dataset, but explains why fine-tuning a pretrained multilingual model is more practical for the current stage.

1 Introduction

Machine Translation is an important application of Natural Language Processing. The goal of machine translation is to automatically convert text from one language into another language while preserving the original meaning. In this project, the focus is on translating Pashto text into English and Hindi.

Pashto is a low-resource language compared to English, Hindi, French, German, or Spanish. A low-resource language has limited availability of clean parallel data, pretrained linguistic tools, benchmark datasets, and high-quality evaluation resources. Because of this, developing an accurate Pashto translation system is more challenging than developing translation systems for high-resource languages.

The current baseline system has already been implemented using the pretrained NLLB model. The system accepts Pashto input text and generates three outputs:

1. Pashto to English translation.
2. Direct Pashto to Hindi translation.
3. Pashto to English to Hindi pivot-based translation.

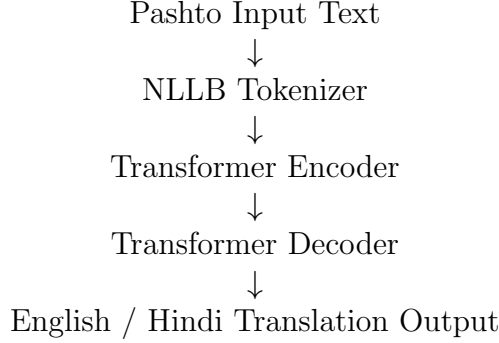
The next goal is to improve the translation quality by studying existing translation resources, collecting datasets, cleaning noisy data, fine-tuning the pretrained model, and evaluating the improvements using standard machine translation metrics.

2 Current Work Completed

In the current stage, a baseline Neural Machine Translation system has been implemented. The pretrained model used is:

`facebook/nllb-200-distilled-600M`

The architecture used by the system is a Transformer-based encoder-decoder sequence-to-sequence architecture. The workflow is:



The language codes used in the current implementation are shown in Table 1.

Table 1: Language codes used in the baseline system

Language	Script	NLLB Code
Pashto	Arabic script	<code>pbt_Arab</code>
English	Latin script	<code>eng_Latn</code>
Hindi	Devanagari script	<code>hin_Deva</code>

The current work includes:

- Python virtual environment setup on Windows.
- Installation of Transformers, PyTorch, SentencePiece, and Streamlit.
- Loading of the pretrained NLLB-200 distilled 600M model.
- Implementation of Pashto-to-English translation.
- Implementation of direct Pashto-to-Hindi translation.
- Implementation of Pashto-to-English-to-Hindi pivot translation.
- Testing of the system on multiple Pashto sentences.
- Initial observation that pivot-based Hindi translation can sometimes preserve more meaning than direct Pashto-to-Hindi translation.

3 Output of the Current Baseline System

The current system successfully generates translations for simple Pashto input sentences. The screenshot below shows the working baseline system.

English Translation:
I'm really learning.

Hindi Direct Translation:
मैं सीख रहा हूँ

Hindi Through English Pivot:
मैं वास्तव में सीख रहा हूँ.

Enter Pashto text: موک هدز وتبې هز

English Translation:
I'm really learning.

Hindi Direct Translation:
मैं सीख रहा हूँ

Hindi Through English Pivot:
मैं वास्तव में सीख रहा हूँ.

Enter Pashto text: مي هېښ هز

English Translation:
I'm fine

Hindi Direct Translation:
मैं ठीक हूँ

Hindi Through English Pivot:
मैं ठीक हूँ

Figure 1: Working output of the baseline Pashto to English and Hindi translation system.

4 Existing Models Available for Pashto Translation

Several pretrained multilingual models can be used for Pashto translation. Since Pashto is a low-resource language, the best starting point is to use a strong multilingual pretrained model rather than training a model from scratch immediately.

4.1 NLLB-200

NLLB stands for No Language Left Behind. It is a multilingual machine translation model developed to support translation across around 200 languages, including many low-resource languages. It is one of the strongest open-source choices for this project because it already supports Pashto, English, and Hindi.

Available NLLB variants include:

- `facebook/nllb-200-distilled-600M`
- `facebook/nllb-200-distilled-1.3B`
- `facebook/nllb-200-3.3B`

The 600M model is smaller and easier to run on a normal laptop. The 1.3B and 3.3B models may provide better quality, but they require more memory and stronger hardware.

4.2 M2M-100

M2M-100 is another multilingual machine translation model that supports many language pairs. It can translate directly between multiple languages without always relying on English as a pivot. However, for this project, NLLB is preferred because it was designed with stronger focus on low-resource translation.

4.3 mBART and mT5

mBART and mT5 are multilingual sequence-to-sequence models. They can be fine-tuned for translation tasks, summarization, and other NLP tasks. They are useful research alternatives, but they may require more task-specific training compared to NLLB for direct translation.

4.4 IndicTrans2

IndicTrans2 is an important model for the Hindi part of this project. Since the target language is Hindi, IndicTrans2 can be used for the English-to-Hindi stage in the pivot pipeline. The improved final pipeline can become:

Pashto $\rightarrow$ English using fine-tuned NLLB
English $\rightarrow$ Hindi using IndicTrans2

This can improve the naturalness of Hindi output compared to direct Pashto-to-Hindi translation.

5 Existing Translation Materials and Dataset Sources

The performance of a machine translation system depends heavily on the quality of the training data. For Pashto, dataset collection and cleaning are extremely important.

5.1 WMT20 Pashto-English Dataset

WMT20 provides Pashto-English data for low-resource parallel corpus filtering and alignment. This is one of the most important sources for this project. However, this data is noisy because it was collected for corpus filtering research. Therefore, it should not be used directly without cleaning.

Possible use:

- Use WMT20 as the main Pashto-English fine-tuning corpus.
- Apply rule-based filtering.
- Apply semantic similarity filtering.
- Use only high-quality sentence pairs for training.

5.2 OPUS Corpus

OPUS is a large collection of open parallel corpora. It contains many parallel datasets across many languages. For this project, OPUS can be used to search for Pashto-English data from sources such as subtitles, web crawls, religious text, educational material, and multilingual corpora.

Possible use:

- Collect additional Pashto-English parallel sentence pairs.
- Compare domains such as subtitles, web text, news, and religious text.
- Merge high-quality OPUS data with WMT20 data.

5.3 FLORES-200

FLORES-200 is useful mainly for evaluation. It should not be treated as the main training data. It provides multilingual evaluation sentences and can be used to test whether the model improves after fine-tuning.

Possible use:

- Use FLORES-200 dev and devtest sets for evaluation.
- Compare baseline NLLB and fine-tuned NLLB.
- Report BLEU and chrF scores.

5.4 Monolingual Pashto Data

Monolingual Pashto data can be collected from sources such as:

- Pashto news websites.
- Pashto Wikipedia pages.
- Public educational text.
- Public government text.
- Public books or open-domain text where licensing allows use.

This monolingual data can be used for back-translation, language modeling, vocabulary analysis, and domain adaptation.

5.5 Manually Created Dataset

A small manually created dataset can also be developed. Even 500 to 2000 high-quality manually checked sentence pairs can be very valuable for evaluation and fine-tuning.

Example categories:

- Daily conversation.
- Education.
- Travel.
- Health.
- Government forms.
- News.
- Technology.
- Emergency communication.

6 Comparison of Available Resources

Table 2: Existing Pashto translation resources and their use in this project

Resource	Type	Use in Project
NLLB-200 distilled 600M	Pretrained model	Current baseline model for Pashto-to-English and Pashto-to-Hindi translation.
NLLB-200 distilled 1.3B	Larger pre-trained model	Future model comparison for better translation quality.
NLLB-200 3.3B	Large pretrained model	Stronger model, but requires high GPU memory and is difficult to run on normal laptop.
M2M-100	Multilingual translation model	Alternative model for comparison.
mBART	Multilingual seq2seq model	Can be fine-tuned, but needs more task-specific training.
mT5	Multilingual text-to-text model	Useful for research comparison, but not the first choice for this project.
IndicTrans2	Indic translation model	Useful for improving English-to-Hindi translation in pivot pipeline.
WMT20	Parallel corpus	Main candidate data source for Pashto-English fine-tuning.
OPUS	Parallel corpus collection	Additional Pashto-English data source.
FLORES-200	Evaluation benchmark	Used for BLEU and chrF evaluation.

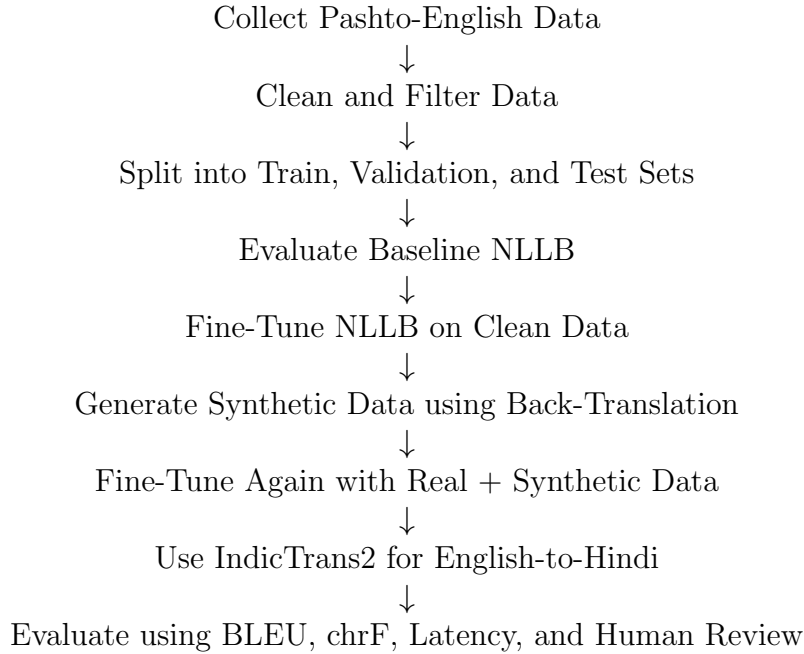
Resource	Type	Use in Project
Manual dataset	Human-created dataset	Small high-quality dataset for testing and error analysis.
Back-translated data	Synthetic dataset	Extra training data for low-resource improvement.

7 Overall Improvement Strategy

The system can be improved in stages. The recommended strategy is not to train from scratch at the beginning. Instead, the best practical approach is:

Pretrained NLLB + Cleaned Dataset + Fine-Tuning + Back-Translation + IndicTrans2 + Evaluation

The complete improvement pipeline is:



8 Step 1: Dataset Collection Plan

The first practical step is to collect all possible Pashto-English data.

8.1 Data Sources

The data can be collected from:

1. WMT20 Pashto-English corpus.
2. OPUS Pashto-English corpora.
3. FLORES-200 evaluation set.
4. Public Pashto news articles.

5. Pashto Wikipedia.
6. Public bilingual websites.
7. Human-created daily conversation dataset.

8.2 Expected Dataset Format

The final dataset should be stored in CSV format:

id	pashto	english
1	source sentence	target sentence
2	source sentence	target sentence

For Hindi evaluation, another file can be created:

id	pashto	english	hindi
1	source	english reference	hindi reference

8.3 Data Licensing

Before using any dataset, the license should be checked. Some datasets can be used for research only, while others may allow broader use. For an academic project, the report should mention the data sources and their usage purpose.

9 Step 2: Dataset Cleaning Plan

Raw data cannot be used directly. Low-resource datasets often contain noise, wrong alignments, wrong language pairs, duplicates, HTML tags, URL noise, and sentence mismatch.

9.1 Basic Cleaning

The first cleaning stage should remove:

- Empty rows.
- Duplicate sentence pairs.
- Sentences with only punctuation.
- Sentences with URLs.
- HTML tags.
- Very short sentence pairs.
- Very long sentence pairs.
- Sentence pairs with extreme length mismatch.

9.2 Language Filtering

Language filtering is required because web-crawled data may contain the wrong language. The system should verify:

- Pashto sentence contains Arabic-script characters.
- English sentence contains Latin-script characters.
- English side is not accidentally Pashto.
- Pashto side is not accidentally English.

9.3 Length Ratio Filtering

A good sentence pair usually has reasonable length similarity. If one side is extremely long and the other side is very short, the pair may be misaligned.

Example rule:

Reject pair if length ratio > 4.0

This means if one sentence is more than four times longer than the other, it may be noisy.

9.4 Semantic Similarity Filtering

After rule-based cleaning, semantic filtering can be applied. The idea is to embed the Pashto sentence and English sentence into a shared multilingual vector space and calculate similarity. If the similarity is low, the pair may not be a correct translation.

Possible tools:

- LASER embeddings.
- LaBSE embeddings.
- Multilingual sentence transformers.

Filtering strategy:

1. Compute embeddings for Pashto sentence.
2. Compute embeddings for English sentence.
3. Calculate cosine similarity.
4. Keep pairs above a threshold.
5. Experiment with thresholds such as 0.65, 0.70, 0.75, and 0.80.

10 Step 3: Dataset Splitting

After cleaning, the dataset should be split into training, validation, and test sets.

Recommended split:

- Training set: 80 percent.
- Validation set: 10 percent.
- Test set: 10 percent.

The test set must not be used during training. It should only be used for final evaluation.

A separate manually checked test set should also be created. This is important because automatic datasets may still contain noise. A manually verified test set of 300 to 1000 sentences can make the evaluation more reliable.

11 Step 4: Baseline Evaluation

Before fine-tuning, the current pretrained NLLB model should be evaluated.

The baseline experiments should include:

1. Pashto to English using NLLB 600M.
2. Direct Pashto to Hindi using NLLB 600M.
3. Pashto to English to Hindi using NLLB 600M.
4. Pashto to English using NLLB 1.3B, if hardware allows.

The baseline output should be saved in CSV format:

id	source	reference	prediction
1	Pashto input	English reference	English output

12 Step 5: Fine-Tuning Plan

Fine-tuning means taking a pretrained model and training it further on our own dataset. This is the most practical approach for this project.

12.1 Why Fine-Tuning is Better than Scratch Training

Fine-tuning is better because:

- The model already understands multilingual translation.
- It already has learned grammar, vocabulary, and sentence structure.
- It requires less data than scratch training.
- It requires less compute.
- It gives faster results.
- It is practical for a student project.

12.2 Fine-Tuning Direction

The first fine-tuning direction should be:

Pashto $\rightarrow$ English

After improving Pashto-to-English, Hindi can be generated using:

Fine-tuned Pashto $\rightarrow$ English + IndicTrans2 English $\rightarrow$ Hindi

12.3 Training Size Experiments

The model should not be trained on all data immediately. Experiments should be done gradually.

Table 3: Fine-tuning experiment plan

Experiment	Dataset Size	Purpose
Exp 1	10k pairs	Check if training pipeline works.
Exp 2	50k pairs	First meaningful fine-tuning.
Exp 3	100k pairs	Better model comparison.
Exp 4	300k pairs	Stronger result if hardware allows.
Exp 5	500k+ pairs	Advanced experiment.

12.4 Full Fine-Tuning

In full fine-tuning, all model parameters are updated. This can give good results but requires more GPU memory.

Advantages:

- Strong adaptation to Pashto-English data.
- Better performance if enough data and compute are available.

Disadvantages:

- High GPU memory requirement.
- Slow training.
- Risk of overfitting if data is small.
- Difficult on a normal laptop.

12.5 LoRA Fine-Tuning

LoRA stands for Low-Rank Adaptation. Instead of updating all model weights, it freezes the original model and trains small adapter matrices. This makes fine-tuning cheaper and faster.

Advantages:

- Lower GPU memory.
- Faster training.
- Practical for limited hardware.
- Easy to save and share adapters.

For this project, LoRA fine-tuning is a strong option if GPU resources are limited.

13 Step 6: Back-Translation

Back-translation is a powerful technique for low-resource machine translation. It creates synthetic training data.

The process is:

1. Collect English monolingual sentences.
2. Translate English sentences into Pashto using an English-to-Pashto model.
3. Create synthetic Pashto-English pairs.
4. Add these pairs to real Pashto-English data.
5. Fine-tune the model again.

The final training data becomes:

Real Pashto-English Data + Synthetic Pashto-English Data

Back-translation is useful because Pashto has limited parallel data, but English monolingual data is available in large quantity.

14 Step 7: Hindi Translation Improvement

For Hindi output, three approaches should be compared.

14.1 Approach 1: Direct Pashto to Hindi

This uses NLLB directly:

Pashto $\rightarrow$ Hindi

This is simple but may be less accurate because direct Pashto-Hindi data is limited.

14.2 Approach 2: Pashto to English to Hindi using NLLB

This uses English as a pivot:

Pashto $\rightarrow$ English $\rightarrow$ Hindi

This can preserve meaning better in some cases because Pashto-to-English and English-to-Hindi translation are usually stronger than direct Pashto-to-Hindi.

14.3 Approach 3: Fine-Tuned NLLB + IndicTrans2

This is the recommended final pipeline:

Pashto $\rightarrow$ English using fine-tuned NLLB
English $\rightarrow$ Hindi using IndicTrans2

IndicTrans2 is specifically designed for Indic languages, so it can improve Hindi output quality.

15 Step 8: Evaluation Plan

Evaluation is necessary to prove that the improved model is actually better than the baseline.

15.1 Automatic Evaluation Metrics

The following metrics should be used:

- **BLEU:** Measures word overlap between prediction and reference translation.
- **chrF:** Measures character-level similarity and is useful for morphologically rich languages.
- **COMET:** Neural metric that can better correlate with human judgment, if available.
- **Inference Time:** Time taken to translate one sentence.
- **Tokens per Second:** Speed of generation.

15.2 Human Evaluation

Automatic metrics are not enough. Human evaluation should also be done.

Human evaluation criteria:

- Meaning preservation.
- Grammar quality.
- Fluency.
- Completeness.
- Naturalness of Hindi output.

Suggested rating scale:

1 = very poor, 2 = poor, 3 = acceptable, 4 = good, 5 = excellent

15.3 Error Analysis

The following error types should be identified:

- Missing words.
- Wrong tense.
- Wrong gender.
- Wrong named entity.
- Literal translation.
- Hallucinated words.
- Poor Hindi fluency.
- Wrong meaning in long sentences.

16 Step 9: Model Comparison Plan

The following model comparison table can be used in the final report.

Table 4: Planned model comparison

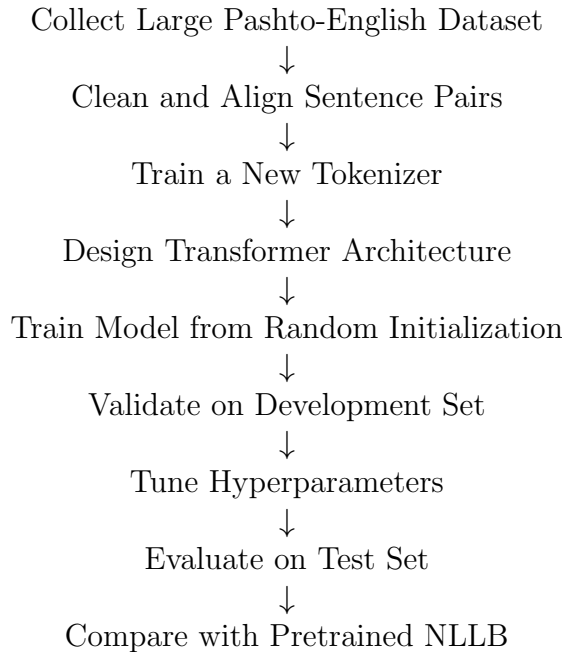
Experiment	Model	Direction	Purpose
1	NLLB 600M baseline	Pashto-English	Current baseline.
2	NLLB 600M fine-tuned	Pashto-English	Check improvement.
3	NLLB 1.3B baseline	Pashto-English	Larger model comparison.
4	NLLB 1.3B LoRA	Pashto-English	Efficient fine-tuning.
5	NLLB direct	Pashto-Hindi	Direct Hindi baseline.
6	NLLB pivot	Pashto-English-Hindi	Pivot comparison.
7	NLLB + IndicTrans2	Pashto-English-Hindi	Best Hindi pipeline.

17 Step 10: Possibility of Training from Scratch

Training from scratch means building a translation model without using pretrained weights. In this case, the model begins with random parameters and learns translation only from the dataset we provide.

17.1 Scratch Training Pipeline

The complete scratch training pipeline would be:



17.2 Dataset Requirement for Scratch Training

Training from scratch requires much more data than fine-tuning. For good translation quality, the dataset should ideally contain hundreds of thousands to millions of high-quality sentence pairs.

Minimum practical levels:

Table 5: Approximate data requirement for scratch training

Dataset Size	Expected Quality	Comment
10k pairs	Very weak	Useful only for demo.
50k pairs	Weak	May memorize patterns.
100k pairs	Basic	Still limited.
500k pairs	Moderate	Possible research baseline.
1M+ pairs	Better	Useful if data is clean.
5M+ pairs	Stronger	Requires serious compute.

17.3 Tokenizer Training

A scratch model needs a new tokenizer. The tokenizer should handle Pashto Arabic script and English Latin script. A SentencePiece tokenizer can be trained using both source and target text.

Tokenizer options:

- Byte Pair Encoding.
- Unigram language model.
- SentencePiece tokenizer.
- Joint source-target tokenizer.

Recommended vocabulary size:

- 16k vocabulary for small experiments.
- 32k vocabulary for standard NMT.
- 64k vocabulary for larger multilingual setup.

17.4 Scratch Model Architecture

A simple Transformer model can be trained from scratch.

Possible architecture:

- Encoder layers: 6.
- Decoder layers: 6.
- Hidden size: 512.
- Attention heads: 8.
- Feed-forward dimension: 2048.
- Dropout: 0.1.

For a larger model:

- Encoder layers: 12.
- Decoder layers: 12.

- Hidden size: 768.
- Attention heads: 12.
- Feed-forward dimension: 3072.

17.5 Compute Requirement for Scratch Training

Training from scratch is expensive.

Challenges:

- Requires large clean dataset.
- Requires GPU training.
- Takes more time.
- Needs hyperparameter tuning.
- Quality may still be worse than fine-tuned NLLB.

For a student project, scratch training is possible only as a small experimental baseline. It should not be the main approach unless enough compute and data are available.

17.6 Recommended Use of Scratch Training

The scratch model can be used as an experiment:

- Train a small Transformer from scratch on 50k or 100k pairs.
- Compare it with pretrained NLLB baseline.
- Show that pretrained models perform better in low-resource conditions.
- Use this as a research finding.

This will make the project stronger because it demonstrates why transfer learning is important.

18 Step 11: Creating a New Dataset

A new dataset can be created to improve the model. The process should be systematic.

18.1 Dataset Categories

The dataset should cover multiple domains:

- Daily conversation.
- Education.
- Healthcare.
- Travel and transport.
- Government services.
- News.

- Technology.
- Emergency and disaster communication.
- Social communication.
- Formal letters and applications.

18.2 Manual Translation Workflow

A high-quality manual dataset can be created using this workflow:

1. Collect English sentences from public-domain sources.
2. Translate them into Pashto using human translators or verified native speakers.
3. Review translations by a second person.
4. Store the pairs in CSV format.
5. Add metadata such as domain and difficulty level.

18.3 Human Review Process

Each sentence pair should be checked for:

- Correct meaning.
- Correct grammar.
- Natural Pashto usage.
- Correct named entities.
- No missing content.
- No hallucinated content.

18.4 Dataset Metadata

The dataset should include metadata:

id	domain	source	target	quality
1	education	English	Pashto	verified

Useful metadata fields:

- Sentence id.
- Domain.
- Source language.
- Target language.
- Translator id.
- Reviewer id.
- Quality score.
- License.

19 Step 12: Data Augmentation Techniques

Data augmentation can increase training data.

19.1 Back-Translation

Back-translation creates synthetic source sentences from target sentences. For example, English text can be translated into Pashto and then used as synthetic Pashto-English training data.

19.2 Forward Translation

Pashto monolingual text can be translated into English to create pseudo-parallel data. This data should be filtered carefully because machine-generated translations may contain errors.

19.3 Paraphrasing

English sentences can be paraphrased and then paired with Pashto translations. This increases sentence diversity.

19.4 Noise Injection

Small noise can be added during training to make the model robust. Examples:

- Random punctuation variation.
- Minor spacing variation.
- Controlled spelling variation.

This should be used carefully because too much noise can reduce quality.

20 Step 13: Domain Adaptation

Translation quality improves when the model is trained for a specific domain. For example, if the final application is for education, the training data should include educational sentences.

Possible domain-specific systems:

- Pashto-English educational translator.
- Pashto-Hindi government service translator.
- Pashto-English healthcare translator.
- Pashto-English news translator.
- Pashto-Hindi daily conversation translator.

A strong final project can include domain labels and allow users to select a domain before translation.

21 Step 14: Multilingual Extension

The user requirement is to eventually translate many languages. This can be done using model routing.

21.1 Language Detection

Before translation, the system should detect the input language.

Possible tools:

- FastText language identification.
- NLLB language identification model.
- Rule-based script detection.

21.2 Model Router

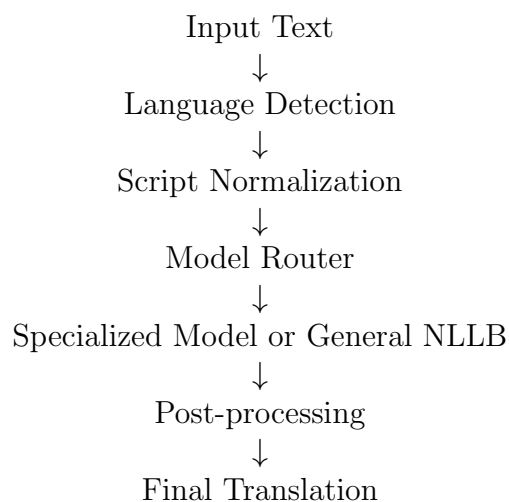
A model router decides which model to use.

Example:

- If source is Pashto and target is English, use fine-tuned NLLB.
- If source is English and target is Hindi, use IndicTrans2.
- If source is Hindi and target is English, use IndicTrans2 or NLLB.
- If language pair is not specialized, use general NLLB.

21.3 Universal Translation Pipeline

The final multilingual architecture can be:



22 Step 15: Deployment Plan

The improved model can be deployed as a web application.

22.1 Streamlit Interface

The first interface can be built using Streamlit. It should contain:

- Source language selector.
- Target language selector.
- Text input box.
- Translate button.
- English output box.
- Hindi output box.
- Option to choose direct or pivot translation.
- Translation history.

22.2 FastAPI Backend

For a more production-ready system, FastAPI can be used.

Backend endpoints:

- `/health`
- `/translate`
- `/languages`
- `/feedback`

22.3 User Feedback

User feedback can be collected:

- Was the translation correct?
- Was the Hindi natural?
- Which output was better: direct or pivot?
- Suggested correction by user.

This feedback can later be used to improve the dataset.

23 Step 16: Practical Timeline

Table 6: Suggested project timeline

Week	Task
Week 1	Complete article on existing models, datasets, and baseline system.
Week 2	Download WMT20 and OPUS Pashto-English data.
Week 3	Implement dataset cleaning and filtering pipeline.
Week 4	Evaluate current NLLB baseline on test data.
Week 5	Fine-tune NLLB on 10k and 50k cleaned sentence pairs.
Week 6	Fine-tune on larger dataset and compare results.
Week 7	Add back-translation and IndicTrans2 Hindi pipeline.
Week 8	Build Streamlit app and final evaluation report.

24 Expected Final Contributions

The final project can claim the following contributions:

1. A working baseline Pashto-to-English and Pashto-to-Hindi translation system.
2. A survey of existing Pashto translation models and resources.
3. A cleaned Pashto-English dataset pipeline.
4. Fine-tuned NLLB model for Pashto-to-English translation.
5. Direct versus pivot Hindi translation comparison.
6. English-to-Hindi improvement using IndicTrans2.
7. Back-translation based data augmentation.
8. Evaluation using BLEU, chrF, inference time, and human analysis.
9. Streamlit-based user interface.
10. A roadmap for multilingual expansion.

25 Risks and Limitations

The main limitations are:

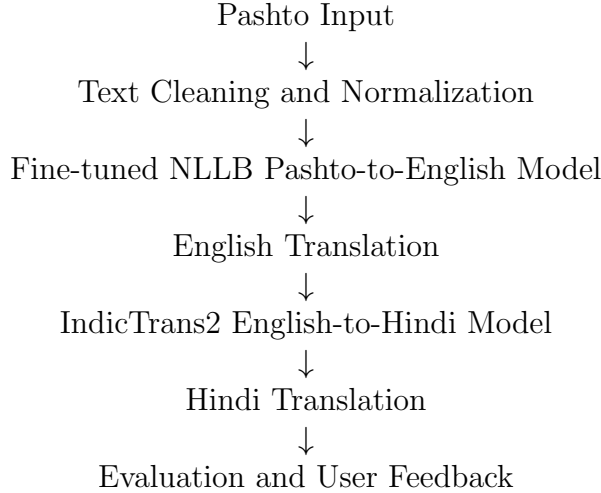
- Pashto data may be noisy.
- Direct Pashto-Hindi parallel data is limited.
- Automatic metrics may not fully capture translation quality.
- Native speaker evaluation may be required.
- Larger NLLB models may need GPU resources.
- Training from scratch may not outperform pretrained models.
- Dataset licensing must be checked before release.

26 Recommended Final Approach

The best practical approach for this project is:

Fine-tuned NLLB for Pashto-to-English + IndicTrans2 for English-to-Hindi

The recommended final system is:



Training from scratch should be included only as an experimental comparison, not as the main approach.

27 Conclusion

This article presented a complete plan for improving Pashto-to-English and Pashto-to-Hindi neural machine translation. The current baseline system already works using the pretrained NLLB-200 distilled 600M model. The next stage should focus on dataset collection, dataset cleaning, baseline evaluation, fine-tuning, back-translation, Hindi improvement through IndicTrans2, and proper evaluation.

The possibility of training from scratch was also discussed. Although scratch training is possible, it requires a large high-quality dataset and strong GPU resources. Therefore, for the current project, fine-tuning a pretrained multilingual model is the most practical and effective approach. A scratch-trained Transformer can be added as a small research baseline to demonstrate the importance of transfer learning in low-resource machine translation.

References

- [1] Meta AI, “No Language Left Behind,” available at: <https://ai.meta.com/research/no-language-left-behind/>
- [2] Hugging Face, “facebook/nllb-200-distilled-600M,” available at: <https://huggingface.co/facebook/nllb-200-distilled-600M>
- [3] Hugging Face Transformers Documentation, “NLLB,” available at: https://huggingface.co/docs/transformers/en/model_doc/nllb
- [4] WMT 2020, “Parallel Corpus Filtering and Alignment for Low-Resource Conditions,” available at: <https://www.statmt.org/wmt20/parallel-corpus-filtering.html>

- [5] OPUS, “Open Parallel Corpus,” available at: <https://opus.nlpl.eu/>
- [6] Meta AI, “FLORES-200,” available at: <https://github.com/facebookresearch/flores/blob/main/flores200/README.md>
- [7] AI4Bharat, “IndicTrans2,” available at: <https://github.com/AI4Bharat/IndicTrans2>
- [8] Hu et al., “LoRA: Low-Rank Adaptation of Large Language Models,” available at: <https://arxiv.org/abs/2106.09685>